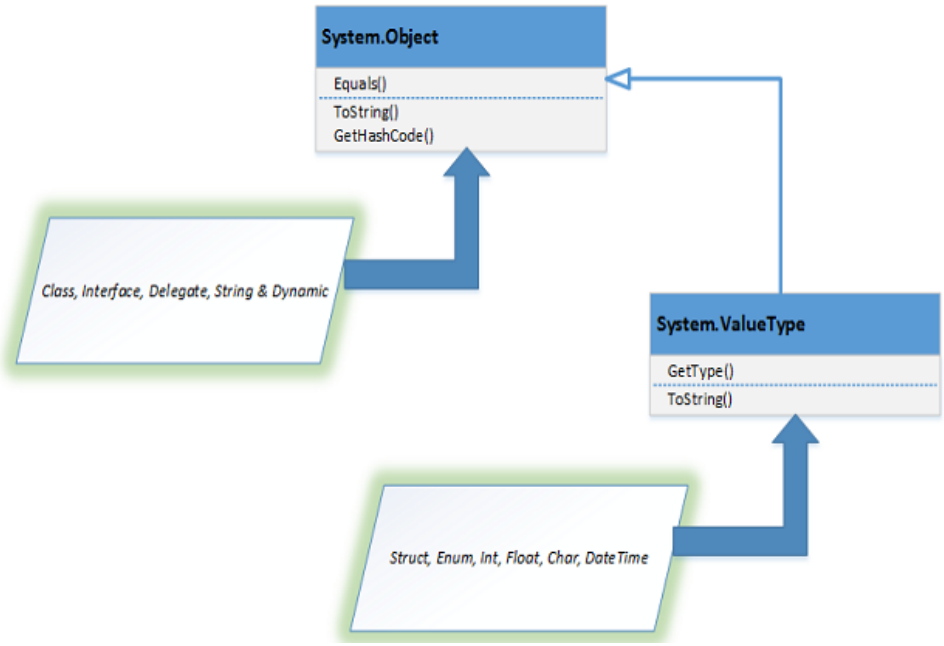
Lecture 10

Reference Types in C#

Types in Microsoft .Net can be either **value type** or **reference type**. While value types are stored generally in the stack, reference types are stored in the managed heap.



- A **value type** derives from System.ValueType and contains the data inside its own memory allocation. In other words, variables or objects or value types have their own copy of the data.
- All fundamental data types, Boolean, Date, structs, and enums are examples of value types.
- A **reference type**, meanwhile, extends System.Object and points to a location in the memory that contains the actual data.
- Examples of reference types include: strings, arrays, objects of classes, etc.

Enums in C#

An **enum** is a special "class" that represents a group of **constants** (unchangeable/read-only variables).

To create an enum, use the enum keyword (instead of class or interface), and separate the enum items with a comma:

Example:

```
using System;

namespace MyApplication
{
  enum Level
  {
    Low,
    Medium,
    High
  }
  class Program
  {
    static void Main(string[] args)
    {
      Level myVar = Level.Medium;
      Console.WriteLine(myVar);
    }
  }
}
```



Enum is short for "enumerations", which means "specifically listed".

Enum inside a Class

You can also have an enum inside a class:

Example

```
class Program
{
 enum Level
 {
  Low,
  Medium,
  High
 }
 static void Main(string[] args)
 {
  Level myVar = Level.Medium;
  Console.WriteLine(myVar);
 }
}
```

Output?

Enum Values

By default, the first item of an enum has the value 0. The second has the value 1, and so on.

To get the integer value from an item, you must explicitly convert the item to an int:

Example

```
enum Months
{
 January,    // 0
 February,   // 1
 March,      // 2
 April,      // 3
 May,        // 4
 June,       // 5
 July        // 6
}


static void Main(string[] args)
{
 int myNum = (int) Months.April;
 Console.WriteLine(myNum);
}
```

The output will be:

```
3
```

Enum in a Switch Statement

Enums are often used in switch statements to check for corresponding values:

Example

```
enum Level
{
```

```
  Low,
  Medium,
  High
}

static void Main(string[] args)
{
  Level myVar = Level.Medium;
  switch(myVar)
  {
    case Level.Low:
      Console.WriteLine("Low level");
      break;
    case Level.Medium:
       Console.WriteLine("Medium level");
      break;
    case Level.High:
      Console.WriteLine("High level");
      break;
  }
}
```

The output will be:

```
Medium level
```

C# Abstraction

Abstract Classes and Methods

Data **abstraction** is the process of hiding certain details and showing only essential information to the user.
Abstraction can be achieved with either **abstract classes** or **interfaces**

The abstract keyword is used for classes and methods:

- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the derived class (inherited from).

An abstract class can have both abstract and regular methods:

```
abstract class Animal
{
 public abstract void animalSound();
 public void sleep()
 {
   Console.WriteLine("Zzz");
 }
}
```

From the example above, it is not possible to create an object of the Animal class:

```
Animal myObj = new Animal(); // Will generate an error (Cannot create an instance of the abstract class or interface 'Animal')
```

Complete Example

```
// Abstract class
abstract class Animal
{
```

```
  // Abstract method (does not have a body)
  public abstract void animalSound();
  // Regular method
  public void sleep()
  {
    Console.WriteLine("Zzz");
  }
}

// Derived class (inherit from Animal)
class Pig : Animal
{
  public override void animalSound()
  {
    // The body of animalSound() is provided here
    Console.WriteLine("The pig says: wee wee");
  }
}

class Program
{
  static void Main(string[] args)
  {
    Pig myPig = new Pig(); // Create a Pig object
    myPig.animalSound();  // Call the abstract method
    myPig.sleep();  // Call the regular method
```

```
  }

}
```

C# Interface

Another way to achieve abstraction in C#, is with interfaces.

An interface is a completely "**abstract class**", which can only contain abstract methods and properties (with empty bodies):

Example

```
// interface

interface Animal

{

  void animalSound(); // interface method (does not have a body)

  void run(); // interface method (does not have a body)

}
```

- By default, members of an interface are abstract and public.
- Interfaces can contain properties and methods, but not fields.

To access the interface methods, the interface must be "implemented" (like inherited) by another class. To implement an interface, use the : symbol (just like with inheritance). The body of the interface method is provided by the "implement" class. Note that you do not have to use the override keyword when implementing an interface:

Example

```
// Interface

interface IAnimal

{

  void animalSound(); // interface method (does not have a body)

}
```

```
// Pig "implements" the IAnimal interface
class Pig : IAnimal
{
  public void animalSound()
  {
    // The body of animalSound() is provided here
    Console.WriteLine("The pig says: wee wee");
  }
}

class Program
{
  static void Main(string[] args)
  {
    Pig myPig = new Pig();  // Create a Pig object
    myPig.animalSound();
  }
}
```

C# Exceptions handling

C# Exceptions

- When executing C# code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.

- When an error occurs, C# will normally stop and generate an error message. The technical term for this is: C# will throw an **exception** (throw an error).

C# try and catch

- The try statement allows you to define a block of code to be tested for errors while it is being executed.

- The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.

The try and catch keywords come in pairs:

Syntax

```
try
{
  // Block of code to try
}
catch (Exception e)
{
  // Block of code to handle errors
}
```

In the following example, we use the variable inside the catch block (e) together with the built-in Message property, which outputs a message that describes the exception:

Example

```
try
{
  int[] myNumbers = {1, 2, 3};
  Console.WriteLine(myNumbers[10]);
}
catch (Exception e)
{
  Console.WriteLine(e.Message);
}
```

The output will be:

```
Index was outside the bounds of the array.
```

You can also output your own error message instead of e.Message.

Finally

The finally statement lets you execute code, after try...catch, regardless of the result:

Example

```
try
{
  int[] myNumbers = {1, 2, 3};
  Console.WriteLine(myNumbers[10]);
}
catch (Exception e)
{
  Console.WriteLine("Something went wrong.");
}
finally
{
  Console.WriteLine("The 'try catch' is finished.");
}
```

The output will be:

```
Something went wrong.
The 'try catch' is finished.
```

The throw keyword

The throw statement allows you to create a custom error.

The throw statement is used together with an **exception class**. There are many exception classes available in C#: ArithmeticException, FileNotFoundException, IndexOutOfRangeException, TimeOutException, etc:

Example

```
static void checkAge(int age)
{
 if (age < 18)
 {
  throw new ArithmeticException("Access denied - You must be at least 18 years old.");
 }
 else
 {
  Console.WriteLine("Access granted - You are old enough!");
 }
}

static void Main(string[] args)
{
 checkAge(15);
}
```

The error message displayed in the program will be:

```
System.ArithmeticException: 'Access denied - You must be at least 18 years old.'
```

C# Namespaces

Namespaces in C# are used to organize too many classes so that it can be easy to handle the application.

In a simple C# program, we use System.Console where System is the namespace and Console is the class. To access the class of a namespace, we need to use namespacename.classname. We can use **using** keyword so that we don't have to use complete name all the time.

In C#, global namespace is the root namespace. The global::System will always refer to the namespace "System" of .Net Framework.

C# namespace example

Let's see a simple example of namespace which contains one class "Program".

```
using System;
namespace ConsoleApplication1
{
   class Program
   {
      static void Main(string[] args)
      {
         Console.WriteLine("Hello Namespace!");
      }
   }
}
```

Output:

```
Hello Namespace!
```

C# namespace example

Let's see another example of namespace in C# where one namespace program accesses another namespace program.

```
using System;
namespace First {
public class Hello
{
```

```
    public void sayHello() { Console.WriteLine("Hello First Namespace"); }
}
}
namespace Second
{
    public class Hello
    {
        public void sayHello() { Console.WriteLine("Hello Second Namespace"); }
    }
}
public class TestNamespace
{
    public static void Main()
    {
        First.Hello h1 = new First.Hello();
        Second.Hello h2 = new Second.Hello();
        h1.sayHello();
        h2.sayHello();

    }
}
```

Output:

```
Hello First Namespace
Hello Second Namespace
```

C# namespace example: by using keyword

Let's see another example of namespace where we are using "using" keyword so that we don't have to use complete name for accessing a namespace program.

```
using System;
using First;
using Second;
namespace First {
```

```
public class Hello
{
   public void sayHello() { Console.WriteLine("Hello Namespace"); }
}
}
namespace Second
{
   public class Welcome
   {
      public void sayWelcome() { Console.WriteLine("Welcome Namespace"); }
   }
}
public class TestNamespace
{
   public static void Main()
   {
      Hello h1 = new Hello();
      Welcome w1 = new Welcome();
      h1.sayHello();
      w1.sayWelcome();
   }
}
```

Output:

```
Hello Namespace
Welcome Namespace
```

C# Generics

- C# allows you to define generic classes, interfaces, abstract classes, fields, methods, static methods, properties, events, delegates, and operators using the type parameter and without the specific data type.
- A type parameter is a placeholder for a particular type specified when creating an instance of the generic type.

- A generic type is declared by specifying a type parameter in an angle brackets after a type name, e.g. `TypeName<T>` where `T` is a type parameter.

Generic Class

Generic classes are defined using a type parameter in an angle brackets after the class name. The following defines a generic class.

Example: Define Generic Class

```
class DataStore<T>
{
   public T Data { get; set; }
}
```

Above, the `DataStore` is a generic class. `T` is called type parameter, which can be used as a type of fields, properties, method parameters, return types in the `DataStore` class. For example, `Data` is generic property because we have used a type parameter `T` as its type instead of the specific data type.

Note

It is not required to use `T` as a type parameter. You can give any name to a type parameter. Generally, `T` is used when there is only one type parameter. It is recommended to use a more readable type parameter name as per requirement like `TSession`, `TKey`, `TValue` etc.

You can also define multiple type parameters separated by a comma.

Example: Generic Class with Multiple Type Parameters

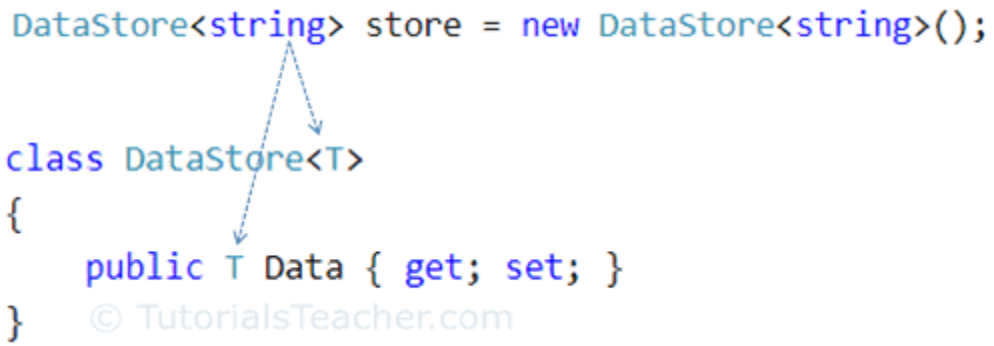
```
class KeyValuePair<TKey, TValue>
{
   public TKey Key { get; set; }
   public TValue Value { get; set; }
}
```

Instantiating Generic Class

You can create an instance of generic classes by specifying an actual type in angle brackets. The following creates an instance of the generic class `DataStore`.

```
DataStore<string> store = new DataStore<string>();
```

Above, we specified the string type in the angle brackets while creating an instance. So, T will be replaced with a string type wherever T is used in the entire class at compile-time. Therefore, the type of Data property would be a string.



You can assign a string value to the Data property. Trying to assign values other than string will result in a compile-time error.

```
DataStore<string> store = new DataStore<string>();
store.Data = "Hello World!";
//obj.Data = 123; //compile-time error
```

You can specify the different data types for different objects, as shown below.

Example: Generic class

```
DataStore<string> strStore = new DataStore<string>();
strStore.Data = "Hello World!";
//strStore.Data = 123; // compile-time error

DataStore<int> intStore = new DataStore<int>();
intStore.Data = 100;
//intStore.Data = "Hello World!"; // compile-time error

KeyValuePair<int, string> kvp1 = new KeyValuePair<int, string>();
kvp1.Key = 100;
kvp1.Value = "Hundred";

KeyValuePair<string, string> kvp2 = new KeyValuePair<string, string>();
kvp2.Key = "IT";
kvp2.Value = "Information Technology";
```

Generic classes encapsulate operations that are not specific to a particular data type. The most common use for generic classes is with collections like linked lists, hash tables, stacks, queues, trees, and so on. Operations such as adding and removing items from the collection are performed in basically the same way regardless of the type of data being stored.

Example:

```
// Declare the generic class.
public class GenericList<T>
{
    public void Add(T input) { }
}
class TestGenericList
{
    private class ExampleClass { }
    static void Main()
    {
        // Declare a list of type int.
        GenericList<int> list1 = new GenericList<int>();
        list1.Add(1);

        // Declare a list of type string.
        GenericList<string> list2 = new GenericList<string>();
        list2.Add("");

        // Declare a list of type ExampleClass.
        GenericList<ExampleClass> list3 = new GenericList<ExampleClass>();
        list3.Add(new ExampleClass());
    }
}
```

OUTPUT?

Another Example to Create List of Integers

```
using System;
using System.Collections.Generic;
// Declare the generic class.
// type parameter T in angle brackets
public class GenericList<T>
{
    // The nested class is also generic on T.
    private class Node
    {
        // T used in non-generic constructor.
        public Node(T t)
        {
            next = null;
            data = t;
        }

        private Node next;
        public Node Next
        {
            get { return next; }
            set { next = value; }
        }

        // T as private member data type.
        private T data;
```

```
    // T as return type of property.
    public T Data
    {
        get { return data; }
        set { data = value; }
    }
}

private Node head;

// constructor
public GenericList()
{
    head = null;
}

// T as method parameter type:
public void AddHead(T t)
{
    Node n = new Node(t);
    n.Next = head;
    head = n;
}

public IEnumerator<T> GetEnumerator()
{
```

```
        Node current = head;

        while (current != null)
        {
            yield return current.Data;
            current = current.Next;
        }
    }
}

class TestGenericList
{
    static void Main()
    {
        // int is the type argument
        GenericList<int> list = new GenericList<int>();

        for (int x = 0; x < 10; x++)
        {
            list.AddHead(x);
        }

        foreach (int i in list)
        {
            System.Console.Write(i + " ");
        }
```

```
        System.Console.WriteLine("\nDone");
    }
}
```

Output :

